Maria Hassan

FYP_Proposal_Finalized.pdf

Quick Submit

Quick Submit

Turnitin

Document Details

Submission ID

trn:oid:::1:6643978461

Submission Date

Oct 28, 2026, 8:47 AM

Download Date

Oct 28, 2026, 8:47 AM

File Name

FYP_Proposal_Finalized.pdf

File Size

497.91 KB

13 Pages

4,058 Words

26,218 Characters

*% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

0 AI-generated only 0%
Likely AI-generated text from a large-language model.

0 AI-generated text that was AI-paraphrased 0%
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.



False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



University of South Asia

Final Year Project Proposal

Identifying the Best Deployable Machine Learning Model for Dengue and Malaria Diagnosis in Low-Resource Settings

Department of Computer Science
Group members: Syed Riyyan Hassan, Muhammad Kamran, Muhammad Awais
Proposed supervisor: Mr. Waqas Ishaq
Submission date: October 2026

Table of Contents

1.1 Project Title 3

1.2 Project Overview Statement 3

1.3 Role and Responsibilities 5

1.4 Project Goals and Objectives 5

1.5 High-Level System Components 6

1.6 List of Optional Functional Units 9

1.7 Exclusions 9

1.8 Application Architecture 10

1.9 Gantt Chart 11

1.10 Hardware and Software Specification 12

1.11 Tools and Technologies Used with Reasoning 12

No Plagiarism and Fair Play Declaration 14

© University of South Asia.

1.1 Project Title

Identifying the Best Deployable Machine Learning Model for Dengue and Malaria Diagnosis in Low-Resource Settings

In this proposal, "deployable" means a model that fits on a cheap phone, works with no internet connection, returns a prediction quickly, and doesn't eat up storage, memory or battery, all without losing its reliability.

1.2 Project Overview Statement

Dengue and malaria are both carried by mosquitoes, and both hit hardest in places where labs are scarce, specialists live far off and the internet can't be counted on. Much of the published machine learning work on these diseases stops at reporting accuracy. Hardly any of it checks whether the model would run on the low-cost phones and basic computers that rural clinics and community health workers really use.

That gap is what this project goes after. We will train six models, three per disease, using two public datasets: a Bangladesh dengue dataset with over 1,500 records in CSV form, and the NIH malaria dataset with over 27,000 labelled red blood cell images (infected and uninfected). For each disease we'll compare two established lightweight models against one we design ourselves. The comparison looks at how well each model predicts, but also at what it costs to deploy: model size, prediction time, memory use, and how much accuracy is lost when the model is compressed. A multi-criteria score then picks one winner for each disease.

I see how both winning models will be placed in an application that runs entirely offline on a phone or, in a web browser. Dengue prediction takes a CSV file. Typed text malaria detection takes an image and a chatbot handles everyday conversation and basic information about the diseases. The application is intended as a screening aid. It is not a substitute for a doctor or a lab test.

Project Overview Statement

Project Title:
Identifying the Best Deployable Machine Learning Model for Dengue and Malaria Diagnosis in Low-Resource Settings

Group Leader:
Syed Riyyan Hassan

Project Members:

Name	Registration	Email Address	Signature
Syed Riyyan Hassan (Leader)	B-29398	b-29398@student.usa.edu.pk	
Muhammad Kamran	B-29197	b-29197@student.usa.edu.pk	
Muhammad Awais	B-28167	b-28167@student.usa.edu.pk	

Project Goal:

To figure out in an repeatable way, which machine learning model works best for real deployment on low-resource devices, for dengue and malaria and to show that choice within an offline diagnostic-support application.

Objectives:

Sr. #	Objective
1	Prepare both datasets for modelling with documented cleaning steps and fixed train, validation and test splits.
2	Train and evaluate three dengue models (two lightweight, one custom).
3	Train and evaluate three malaria models (two lightweight, one custom).
4	Measure the deployment cost of all six models after export to a mobile-friendly format.
5	Rank the models of each disease with a multi-criteria Deployability Score and select one per disease.
6	Build a mobile and web application. Test the mobile and web application. The offline mobile and web application should use the selected models and a chatbot.

Project Success criteria:

- All six models are. Evaluated using the same test data that is not used during training and all the results are shared openly.
- For each disease the model that is chosen is within 2 percentage points of the F1-score among the three models being tested and it stays under 10 MB for malaria and under 2 MB for dengue.
- For malaria the model that is selected achieves at 95% accuracy and 95% recall for the infected class (research on this dataset often shows results in the high 90s). The dengue model will be finalized after the baseline run because the dataset is small.
- Predictions take than one second on the test phone.
- The app works when the network is off and every test input, like an image, a CSV file, typed text or a chat message is sent to the correct part of the app.

Assumptions, Risks and Obstacles:

Assumptions. Both datasets are public and can be used for academic work. We take their labels to be correct. At least one mid-range Android phone will be available for testing.

Risks. The dengue dataset is small, so models could overfit and scores could shift from one split to another (we'll use cross-validation and report the spread rather than one number). The malaria images all come from one imaging setup. That means accuracy might go down when using photos, from microscopes or phone cameras. It’s not the equipment so results can vary. I’ve seen this happen before. Different lighting, focus or sensors change how the images look. So even if the model works here it might not work as well elsewhere. That’s something to keep in mind. ONNX models running in a browser engine may be slower than native code on weak phones (fallback: a native Android runtime). Free GPU time is also limited.

Obstacles. Clinical validation is not possible within this project. The results describe the datasets we used and should not be taken as clinical performance.

Organization Address (if any):

Department of Computer Science, University of South Asia

Type of project:

Research

© University of South Asia.

Target End users: Community health workers and clinic staff in low-resource areas (primary). Students and researchers who want to see how the models compare (secondary).
Development Technology: Object Oriented
Platform: Web based Other: Android mobile application (offline)
Suggested Project Supervisor: Mr. Waqas Ishaq
Approved By:
Date:

1.3 Role and Responsibilities

The roles below are only a first split. We'll confirm them with the supervisor, and members will help each other out wherever the workload is lopsided.

Roles	Name and Responsibilities
Supervisor Coordinator	Syed Riyyan Hassan is the contact with the supervisor. Syed Riyyan Hassan schedules and records progress meetings compares progress, with the Gantt chart and flags blockers early.
Requirement Engineer	Muhammad Kamran writes the non-functional requirements. Muhammad Kamran keeps track of dataset sources and licences. Muhammad Kamran prepares test cases that link back, to each requirement.
Analysis	Muhammad Kamran works on dengue and Syed Riyyan Hassan works on malaria. We conduct exploratory data analysis, preprocessing, training, tuning and metric calculation. Syed Riyyan Hassan and Muhammad Kamran share the TOPSIS ranking.
Designer	Syed Riyyan Hassan is responsible, for screen design result layout, input router and chatbot conversation flow. Syed Riyyan Hassan works on screen design. Syed Riyyan Hassan handles result layout. Syed Riyyan Hassan manages input router. Syed Riyyan Hassan creates chatbot conversation flow.
Coding	Syed Riyyan Hassan worked on the application using React ONNX Runtime Web and an Android wrapper. Muhammad Kamran handled the dengue pipeline and the chatbot. Muhammad Awais focused on the malaria pipeline, ONNX export and device benchmarking.
Documentation	I see that every member writes up their part. Muhammad Awais edits the report and puts together the presentation.

© University of South Asia.

1.4 Project Goals and Objectives

Goals

Our first goal is to stop judging models by accuracy and also consider what it costs to run them on a low-end device. Our second goal is to give health workers something they can actually use: one application that accepts what they already have on hand such as a patient record or a blood smear image and returns a prediction along, with a clear reminder that it is only a screening aid

Objectives

Every objective has a way to measure it so we can truly judge progress when the project ends of just saying what happened.

Sr. #	Objective	How it will be measured
1	Prepare both datasets for modelling.	Written cleaning steps. The train splits, the validation. The test splits have been fixed. These splits were saved using a seed. For images the splits are made by slide using the file names when that is possible. This ensures that cells, from one patient do not appear on both sides.
2	Train three dengue models. Two lightweight, one custom.	Accuracy, precision, recall, F1 score and ROC-AUC on the held-out test set. Mean and standard deviation, across -validation folds.
3	Train three malaria models. Two lightweight, one custom.	I keep the metrics. I report recall for the class alone because missing an infection in the infected class is worse, than raising a false alarm.
4	Measure deployment cost of all six models.	File size in megabytes, latency, the 95th percentile latency measured in milliseconds and peak memory consumption in megabytes on a mid-range Android phone plus the change in F1 score, after applying 8-bit quantisation.
5	Rank. Select one, per disease.	The TOPSIS Deployability Score uses weights and a ranking table. The TOPSIS Deployability Score also includes a sensitivity check that changes each weight by twenty percent to see if the winner changes.
6	Test the offline application.	All tests work when the network is turned off, on least two devices. Router tests include image, CSV, typed text and chat inputs, including inputs that have the format.

© University of South Asia.

1.5 High-Level System Components

The system has six main components. The first three make up the study, which runs on a laptop or a free cloud GPU. The last three make up the application that runs on the phone or in the browser.

Component 1: Data preparation

Inputs	Dengue CSV file from Bangladesh with than 1,500 records. NIH malaria images with, than twenty-seven thousand labelled parasitized or uninfected.
Processing	Dengue: look for missing values and duplicates encode categorical columns, scale numeric columns, check class balance and apply class weights or resampling to the training part. Malaria: resize to one fixed size (96 x 96 pixels, as a starting point tested against 64 and 128) scale pixel values and apply augmentation (flips, small rotations, brightness changes) to training images only.
Outputs	Ready-to-train splits, along with saved scalers and encoders. The application uses these saved objects again. The preprocessing on the phone is the same, as what happened during training.

Component 2: Model training and evaluation

Inputs	I have prepared the splits, from Component 1.
Processing	We will train the six models that are listed in the table. I will tune the settings using the validation data or by cross-validation. The test set will be used once at the very end.
Outputs	Trained models, metric tables, confusion matrices and ROC curves.

Component 3: Benchmarking and model selection.

Inputs	The six trained models.
Processing	Export each model to ONNX format. Apply 8-bit quantisation wherever it is possible. Check the size of each model. Check the latency of each model. Check the memory usage of each model. Check the change in accuracy of each model. Do this on the test phone. Then calculate the Deployability Score for each disease. Use TOPSIS to calculate the Deployability Score for each disease.
Outputs	Benchmark table, ranking and the model that was chosen for each disease.

Component 4: On-device inference and input router

Inputs	An image, whether it comes from a file or a camera a CSV file, typed text or a chat message.
Processing	Images go to the malaria model. A CSV file or text typed as field and value pairs, on the dengue screen goes to the dengue model after its columns are checked. Chat messages go to the chatbot. If the input is incorrect or cannot be understood the user receives a message. Nothing is sent to the model.

© University of South Asia.

Outputs	Provide a guess and a confidence score and then an error message that tells the user exactly what must be corrected.

Component 5: User interface

Inputs	Input from the user, through touch or keyboard.
Processing	The app has screens, for the home page, dengue screening, malaria screening, chat, medical history and an About page that includes the disclaimer. One codebase runs on both the browser and the Android wrapper.
Outputs	There's a result card that shows the label the confidence level, a note advising the user to visit a clinician and the disclaimer.

Component 6: Chatbot

Inputs	Messages typed in the chat screen.
Processing	Checking if the users message matches any of the questions and answers that were prepared. These questions and answers are about symptoms, how to prevent issues when to get help how to use the app and regular conversations. The chatbot does not provide any diagnosis. If a user says they want to be tested the chatbot sends them to the screening pages.
Outputs	A brief reply, in text.

Candidate models

The two existing models chosen for each disease might change after some small pilot runs, in consultation with the supervisor. We design and tune the custom models ourselves.

Disease	Model	Type	Why it is included
Dengue	Logistic Regression	Existing	It is very small and fast. A reasonable baseline, for tabular data
Dengue	LightGBM	Existing	Gradient-boosted trees often work well on tables. They also stay compact in size.
Dengue	Custom shallow neural network	Custom	We use a feed-forward network. I pick the number and width of layers by searching on validation data. I also use a feed-forward network to keep things simple.
Malaria	MobileNetV3	Existing	Designed for use. The pretrained weights were fine-tuned on cell images for use.

© University of South Asia.

Disease	Model	Type	Why it is included
Malaria	ShuffleNetV2	Existing	We have a mobile-oriented network that is a mobile-oriented network, with a very small footprint.
Malaria	Custom compact CNN	Custom	A few depthwise-separable convolution blocks, aiming for, than 0.5 million parameters.

Evaluation and selection criteria

TOPSIS ranks each model by how close it's to the best value on every criterion and how far it is from the worst value. Values are first normalised, so criteria measured in units such, as percent, megabytes and milliseconds can be combined. The weights listed below are a starting point giving predictive quality and deployability equal importance. We will go over them with the supervisor. Test them in the sensitivity check.

Criterion	Direction	Weight	Group
F1-score	Higher is better	0.20	Predictive
Recall (sensitivity)	Higher is better	0.20	Predictive
ROC-AUC	Higher is better	0.10	Predictive
Model file size (MB)	Lower is better	0.15	Deployment
Inference latency (ms)	Lower is better	0.15	Deployment
Peak memory (MB)	Lower is better	0.10	Deployment
F1 drop after quantisation	Lower is better	0.10	Deployment

1.6 List of Optional Functional Units

These features are worth building if time and skills allow. The core system works without features with fewer conveniences.Result history stored on the device, with export to CSV.

- Explanation views: a heat map over the cell image (Grad-CAM) for malaria and a feature-importance chart for dengue.
- English and Urdu interface so users can choose their language.
- An uncertainty rule: if confidence is low the app asks for a retest or a clinician visit of showing a label.
- Batch mode for dengue, where a CSV file containing patients returns a summary table that can be saved.
- Camera capture for malaria with a quality check (blur and brightness) before prediction.
- A small quantised language model, for the chatbot used only if it stays within the size and speed limits.

© University of South Asia.

Non-functional targets

- **Performance:** prediction in under one second on the test phone. Application package under 50 MB.
- **Usability:** a first-time user can complete one screening in under two minutes with no training.
- **Privacy:** no patient data ever leaves the device, since the application doesn't send anything over a network.
- **Compatibility:** Android 8 or newer, and current versions of Chrome, Edge and Firefox.

Assumptions and issues

We will keep two lists and go through them with the supervisor at the end of each phase. The assumptions list begins with the items from the overview statement. The issues list begins with three open questions: whether the Bangladesh dataset is big enough for a custom neural network to compete with simpler models, whether the NIH images carry over to phone-camera photos of smears, and whether in-browser inference is fast enough on the weakest test device.

1.7 Exclusions

The following items are outside the scope of this project.

- Clinical trials, hospital deployment and regulatory approval. The application is a research prototype.
- Collecting new patient data. Only the two public datasets are used.
- Malaria parasite species identification, parasite stage detection or thick smear analysis.
- Dengue severity grading or serotype prediction, beyond whatever target label the dataset provides.
- Hardware for capturing images through a microscope.
- An iOS application, user accounts, or any cloud backend.
- Medical advice from the chatbot. It handles only general conversation and basic public information.

1.8 Application Architecture

The application has no server. Everything runs on the user's device, so there is just one physical tier. Logically it is split into three layers. The presentation layer is the React interface. The logic layer contains the input router, the two prediction models and the chatbot. The data layer is the device's local storage, which also holds the bundled model files and the saved preprocessing objects.

Figure 1 shows the full flow. Phase A is the study done during development, and its output, one model per disease, is bundled into the application at build time. Phase B is what the user actually sees on the phone or in the browser.

© University of South Asia.

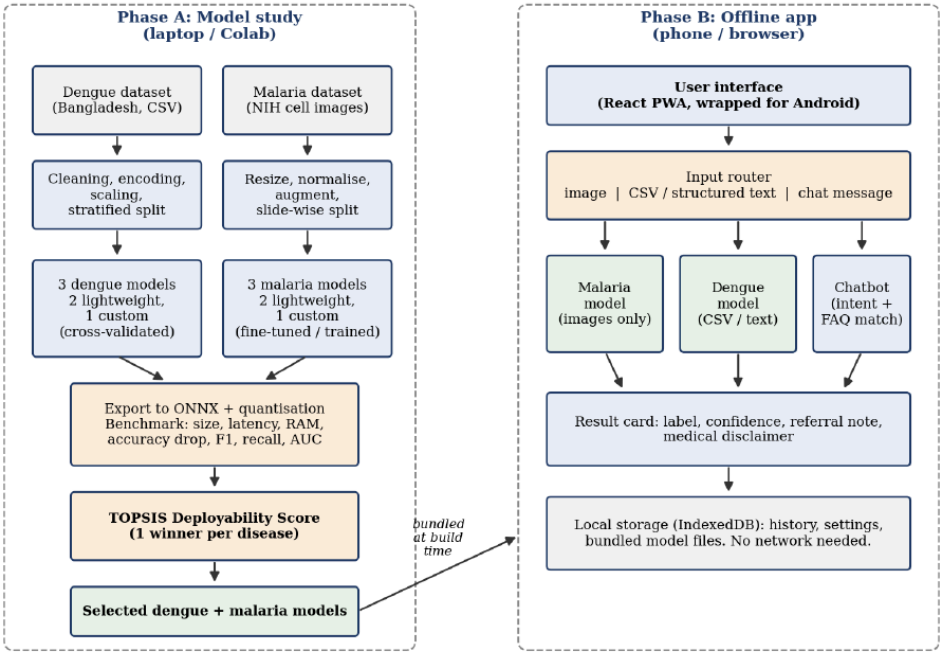


Figure 1. Overall architecture: model study (Phase A) and offline application (Phase B)

The web version is a progressive web app, so a service worker caches the pages and model files and lets it work offline after the first load. The same code is wrapped for Android. If measurements show that in-browser inference is too slow on weak phones, we'll fall back to running the same ONNX models through the native Android runtime.

1.9 Gantt Chart

The plan assumes roughly 32 weeks, which covers both semesters of the project. If the university calendar turns out shorter, the model study stays as it is and the optional units are the first things to go. Responsible members appear as M1 to M3 (see Section 1.3).

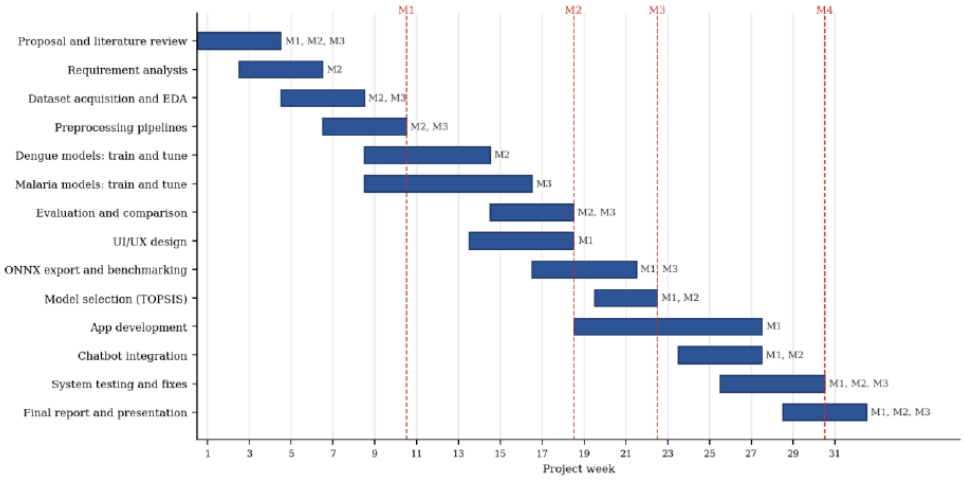


Figure 2. Project schedule by week, with milestones M1 to M4

Milestones

- **M1, week 10:** both datasets prepared and baseline models running.
- **M2, week 18:** all six models trained and evaluated.
- **M3, week 22:** benchmarks complete and one model selected per disease.
- **M4, week 30:** offline application built and tested on two devices.

Duration, resources and dependencies

Activity	Weeks	Responsible	Depends on
Proposal and literature review	1 to 4	All	None
Requirement analysis	3 to 6	M2	Proposal
Dataset acquisition and EDA	5 to 8	M2, M3	Requirements
Preprocessing pipelines	7 to 10	M2, M3	EDA
Dengue models: train and tune	9 to 14	M2	Dengue preprocessing
Malaria models: train and tune	9 to 16	M3	Malaria preprocessing
Evaluation and comparison	15 to 18	M2, M3	Model training
UI/UX design	14 to 18	M1	Requirements
ONNX export and benchmarking	17 to 21	M1, M3	Evaluation (each model as it finishes)
Model selection (TOPSIS)	20 to 22	M1, M2	Benchmarking
App development	19 to 27	M1	UI/UX design
Chatbot integration	24 to 27	M1, M2	App core screens
System testing and fixes	26 to 30	All	App and chatbot

© University of South Asia.

Activity	Weeks	Responsible	Depends on
Final report and presentation	29 to 32	All	Testing (report writing runs from week 1)

1.10 Hardware and Software Specification

Hardware

Item	Specification
Development laptops (one per member)	Quad-core CPU or better, 16 GB RAM (8 GB is workable), 256 GB SSD or larger.
Training GPU	Free cloud GPU sessions (Google Colab or Kaggle) for the malaria models. The dengue models and the chatbot train on a laptop CPU.
Test phone, mid-range	Android 10 or newer, 4 GB RAM. Used for all reported latency and memory figures.
Test phone, low-end (if available)	Android 8 or newer, 2 GB RAM. Used to check the worst case.

Software

Item	Specification
Operating system	Windows 10 or 11, or Ubuntu 22.04, on the development laptops.
Modelling	Python 3.10 or newer, PyTorch and torchvision, scikit-learn, LightGBM, pandas, NumPy, Matplotlib.
Export and inference	ONNX, ONNX Runtime (with its quantisation tools) and ONNX Runtime Web.
Application	Node.js, React, a service worker for offline caching, Capacitor for the Android wrapper, Android Studio for building and device testing.
Collaboration	Git and GitHub, Visual Studio Code, Jupyter notebooks.

1.11 Tools and Technologies Used with Reasoning

The project has no budget, so every tool we use is free. The three of us work partly remotely, which is why all code and experiment logs go through Git. Development is iterative: each cycle ends with a logged experiment or a working build, so a failed idea shows up early instead of in the final month.

Tool	Used for	Reason for choosing it
Python, PyTorch	Training the neural models (custom CNN, custom dense network, fine-tuned MobileNetV3 and ShuffleNetV2)	Pretrained mobile networks are available in torchvision, and exporting to ONNX is easy.
scikit-learn, LightGBM	Logistic Regression, LightGBM, preprocessing, cross-validation	Standard, well-documented tools for small tabular data. Both can export to ONNX.
ONNX and ONNX Runtime	One common model format for all six models; quantisation; device benchmarking	One format lets us compare all six models under the same runtime, and it runs on both Android and the browser.
React and a service worker	Web interface and offline caching	A single codebase covers both the browser and Android versions. Cached assets let the app run with no network.
Capacitor	Packaging the web app as an Android app	Saves us from writing the interface twice. It also gives access to the device camera and file picker.
Android Studio	Building the Android package and testing on phones	The official tool, and its device profiling helps with memory and latency measurements.
Google Colab or Kaggle	GPU training for malaria models	Free GPU access suits a project with no budget.
Git and GitHub	Version control and shared work	Keeps code and results in sync across the three members.

© University of South Asia.

NO PLAGIARISM AND FAIR PLAY DECLARATION

We the group members of the FYP titled "**Identifying the Best Deployable Machine Learning Model for Dengue and Malaria Diagnosis in Low-Resource Settings**" understand the meaning and implications of the act of plagiarism in academic works and we do solemnly declare and promise not to indulge ourselves directly or indirectly in any acts of plagiarism and/or use or misuse of any work done by other parties, or any activities that are considered miss-appropriate by the project advisers/supervisors and/or considered to be illegal by the regulations of any kind; unless of course permitted by our project adviser/supervisor that is within legal bounds and is/are deemed necessary by them. Our project and product are unique, of quality and are not a reputation or copy of any previous project(s). We declare that we will produce work that is genuine, innovative and reflective of all the study that we had as the students of university of south Asia.

We promise to follow the schedule during which we shall seek feedback and maintain a liaison with our adviser. We understand that any foul-play or infringement on our part will result in the cancellation of our project and possibly other penalties may be imposed upon us.

Our project efforts and the end-product are safe, harmless and helpful to humans and society.

Signed:

No.	Member Name	Roll No	Email	Phone	Batch
1	Syed Riyyan Hassan	B-29398	b-29398@student.usa.edu.pk	0331 6562153	2023 - 2027
2	Muhammad Kamran	B-29197	b-29197@student.usa.edu.pk	03099256058	2023 - 2027
3	Muhammad Awais	B-28167	b-28167@student.usa.edu.pk	0320 6638680	2023 - 2027

Project Supervisor Name: Mr. Waqas Ishaq

Signature: _____

Project Coordinator Name: Mr. Zaid-bin-Javaid

Signature: _____

HOD CS Name: Dr. Nadeem Asif

Signature: _____

© University of South Asia.